

Cik Lēns Tu esi?

DatZB011: Datori un programmēšana

Pārsla Bogdanova
(Dated: 2025. gada 21. decembrī)

I. IEVADS

Reakcijas laiks ir laiks, kas paiet no brīža, kad kaut ko uztveram, kad mēs uz to reaģējam. Tas raksturo laika intervālu vizuāla signāla parādīšanos un muskuļu reakcijas uzsākšanu. Ikdienas dzīvē reakcijas laikam (ātrumam) ir svarīga loma tādās aktivitātēs kā bumbas ķersana, šķēršļu izvairīšanās, braukšana (piemēram, Formula 1 sacīkstēs), spēļu spēlēšana un citās situācijās.

Mūsu reakcijas laiku ietekmē gan vizuālā uztvere, gan ķermeņa kustība. Acis var uztvert vizuālo stimulu nedaudz ātrāk, nekā smadzenes var nosūtīt signālu muskuļiem, lai veiktu darbību. Lai gan šī aizture ir tikai 13-17 milisekundes (ms)[1] (tieši tik ilgi mūsu acis nosūta signālu uz smadzenēm), tā kļūst nozīmīga uzdevumos, kuriem nepieciešama augsta precizitāte un ātra reakcija.

Reakcijas laiku parasti mēra vai pārbauda, izmantojot vienkāršus tiešsaistes testus, kuros lietotājs reaģē uz vizuālu signālu, noklikšķinot ar peli vai nospiežot taustiņu (noklikšķinot uz mērķa ekrānā vai ierakstot vārdu skaitu *WPM*.) Šie testi bieži koncentrējas uz veiksmīgu reakciju skaitu vai vidējo reakcijas laiku (vidēji $250ms$). Tomēr tie neņem vērā uzdevuma grūtības pakāpi, mērķa kustību vai ātruma izmaiņas. Piemēram, reālās dzīves situācijā mērķi reti ir statiski un paredzami, un medniekam medībās ir jāreaģē uz kustīgiem mērķiem. Tie var mainīt virzienu un reaģēt uz briesmām.

Galvenais mērķis ir izpētīt reakcijas laiku citādā veidā, izstrādājot pielāgotu reakcijas laika spēli. Šajā spēlē programma ģenerē nejaušu gredzena pozīciju, kamēr apla rādiuss (sākumā) paliek nemainīgs. Pēc pieciem veiksmīgiem trāpījumiem, grūtības pakāpe palielinās, saraujas uz centra virzienu, samazinot rādiusu, padarot to grūtāk trāpīt. Tas ļauj analizēt ne tikai reakcijas ātrumu, bet arī precizitāti, palielinoties grūtības pakāpei. Spēle beidzas pēc maksimāli desmit neveiksmīgiem mēģinājumiem.

Sistēma aprēķina attālumu starp gredzena centru un peles klikšķa pozīciju. Klikšķi ārpus gredzena netiek nekavējoties uzskatīti par neveiksmīgiem mēģinājumiem, ja vien rādiuss nekļūst par ≤ 0 . Tas ļauj detalizētāk analizēt reakcijas precizitāti. Apkopotie dati tiek izmantoti, lai ģenerētu diagrammas, kas parāda “reakcijas laiku”, “attālumu” no gredzena centra un trāpījuma “relatīvais blīvums”.

Relatīvais blīvums - parāda cik “tipisks” vai “bieži” ir noteikta vērtība attiecībā pret visiem datiem. Cik liela datu daļa koncentrējas ap x vērtību.

II. METODES

Lai saprastu, kā veidot spēli tikai ar *Python* programmēšanas valodu, opcijas ir: Pygame vai Unity, Godot.

Pygame ir *Python* bibliotēka, ko izmanto spēļu un cita multimediju lietojumprogrammas izstrādei. Darbojas uz vairākām platformām un izmanto *SDL* (Simple DirectMedia Layer) - starpplatformu izstrādes bibliotēka, kas nodrošina zema līmeņa piekļuvi audio, tastatūrai/klaviatūrai, pelei, spēļu dzoistiki un grafikas sistēmai (2D grafiku, animācijas, skaņu, rezultējot spēļu izveidei). Pygame ir divas versijas[2] [3]:

1. “pygame” - veca versija, nav atjauninājumu.
2. “pygame-ce” - aktīvi uzturēts, moderniskāks un vairāk funkciju, labāka veiktspēja. Instalējot: ‘pip install pygame-ce’. Ieteicams izmantot *Python* jaunāko versiju. Pārbaudot terminālī: ‘python – version’

Lai kods būtu pārskatāms un loģiski sadalīts, klases “Game”, “Circle” un “Analysis” ir visi folderī *OOP* (Object-oriented programming). *main.py* koda sākums [IV](#)

Aplā objekta izveide(Circle.py)

Klase “Circle” satur atribūtus: rādius, krāsu, atrašanās vietu (x, y) uz ekrāna un laiku, kad aplis parādījās. Katru reizi lietotājs noklikšķina iekšējā apla līnijā, jauns aplis uz jaunas pozīcijas atrodas.

1. Metode *spawn()* ģenerē apla koordinātes nejaušā vietā logā (skatoties pēc loga izmēra). Šeit tiek arī saglabāts laiks, kad aplis parādījās, lai vēlāk varētu aprēķināt lietotāja reakcijas laiku.

2. Metode *update()* [IV](#) laika gaitā samazina apla rādiusu [1](#), lineāra samazināšanas laikā. Tas ļauj mainīt objekta izmēru atkarībā no pagājuša laika dt . Nodrošina, ka aplis kļūst grūtāk trāpams. v ir saraušanas ātrums.

$$r_{k+1} = r_k - v_{k+1} \cdot \Delta t \quad (1)$$

3. Metode *draw()* uzzīmē apli uz ekrāna, izmantojot tā pašreizējo rādiusu izmēru un krāsu.

4. Metode *is_clicked()* nosaka, vai lietotāja klikšķis atrodas apla iekšpusē. To aprēķina pēc attāluma formulas starp apla centru un klikšķa koordinātēm [2](#). Pēc tā, tiek ģenerēts jauns gredzens uz ekrāna loga.

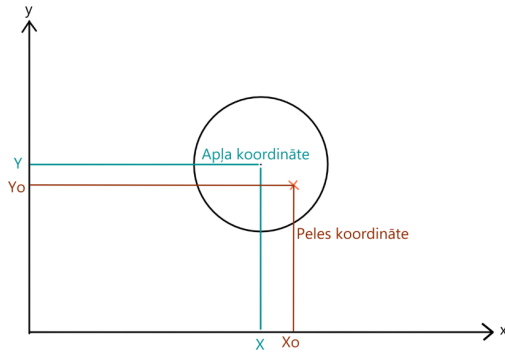
$$d = \sqrt{(x_0 - x)^2 + (y_0 - y)^2}$$

$$d = \sqrt{(x_{\text{mouse}} - x_{\text{ring}})^2 + (y_{\text{mouse}} - y_{\text{ring}})^2} \quad (2)$$

5. Metode *level_up()* iedarbojas, kad +5 veiksmīgi trāpīti mērķi gredzena laukuma iekšpusē. 3. **k** ir līmeņa palielināšanas reižu skaits.

$$v_{k+1} = v_0 + 5\mathbf{k} \quad (3)$$

Klišķis tiek uzskatīts veiksmīgs, ja $d \leq r$. Attāluma pārbaide ir metode, kas nosaka, vai lietotājs trāpījis aplī. 1



Att. 1. Koordinātu sistēmas shēma

Spēles izveide(Game.py)

“Game” klase pārvada spēles gaitu(sākuma, beigu, miss_count, max, game_over), bet “Circle” klase apraksta tikai mērķa īpašības un uzvedību. Klase “Game” importē “Circle”, lai izmantotu to kā spēles objektu.

1. Metode *check_miss()* pārbauda, vai spēlētājs nav trāpījis mērķi vai gredzena rādiuss ir smazinājies līdz nullei. Ja mērķis nav trāpīts, tiek pieskaitīts izlaisto mērķu skaits miss_count par +1 un gredzens tiks ģenerēts no jauna. Kad maksimālo vērtību “max_miss = 10”, rezultātā spēle ir beigusies.

2. Metode *restart()* atsāk izlaisto mērķu skaitu un spēles stāvokli, kā arī restartē objektu koordinātes, atļaujoties spēlei atsākties no sākuma.

Analīze(Analysis.py)

1. Metode *record_clicks()* iegūts datus, kad spēle ir beigusies. Aprēķina attālumu 2 starp klišķi un centra punktu gredzena. Parametrs “reaction_time”-laiks, kas pagājis līdz klišķim. Darbība *append()* pievieno attālumu(distance) un reakcijas laiks(times) sarakstam.

$$\Delta t_{ms} = \frac{16,7ms}{1000} \cong 0,0167s$$

Dzīvē, kur reakcijas laiks iedarbojas uz zemes, formula ir:

$$t = \sqrt{2 \cdot \frac{d}{g}}$$

Bet takā uz video spēlēm nedarbojas gravitācija, formula tiek pārveidota uz šo:

Fizikā:

$$t = f(d, g)$$

Reakcijas ekperiments:

$$t = f(\text{signal}, \text{human})$$

Programmā:

$$\Delta t = t_{\text{mouse_click}} - t_{\text{start}}$$

2. Metode *gauss_line()* ģenerē gausa līnki, kas atbilst datu sadalījumam - normālais sadalījums.

3. Metode *show_results()* attēlo un saglabā vizualizācijas PDF failā. Izsacot “gauss_line” gan “distance”, gan “times”.

Izmantotie resursi (Tools)

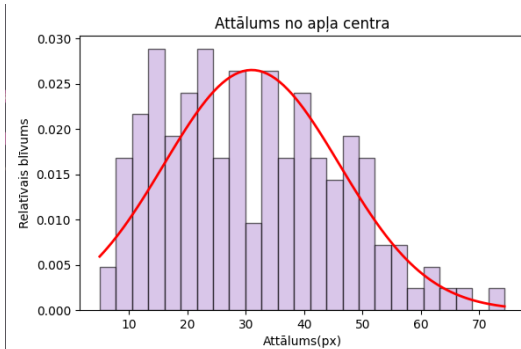
Bibliotēkas

- Numpy - piedāvā matemātiskas fukcijas, nejaušo skaitļu ģeneratorus.
- matplotlib.pyplot - pyplot ir API(Application Programming Interface) priekš Python matplotlib.
- matplotlib.backends.backend_pdf import PdfPages - izmanto matplotlib attēlu attēlošanai, ekrānā vai ierakstīšanai failos.
- scipy.stats import norm - fukcija SciPy valodā, kas aprēķina normāla sadalījuma(Gausa) varbūtības blīvuma funkciju(PDF) pie dotās vērtības x.
- math - matemātiskās fukcijas
- pygame - spēļu veidošana un vizuālā saskarne

Izmantojot metode *update()* IV no klases “Circle”, ar Chatgpt palīdzību, vajadzēja izprast kā animāciju pievienot pie objekta, “shrinking”, samazinot vai palielinot objekta izmēru.

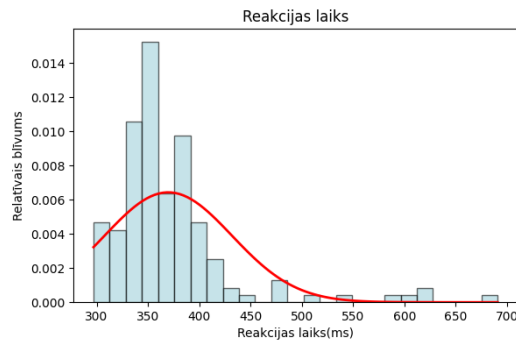
III. REZULTĀTI UN DISKUSIJA

Datu analīze rāda, ka reakcijas laiks un gredzena lauka trāpījuma ir atkarīga no vairākiem faktoriem, tostarp spēles veida, mērķa kustības, kā arī lietotāja atribūtiem(vecums, dzimums, u.c.). Lai gan faktiskie rezultāti atšķiras atkarībā no spēles grūtības un koordinātu izvietojuma. [4]



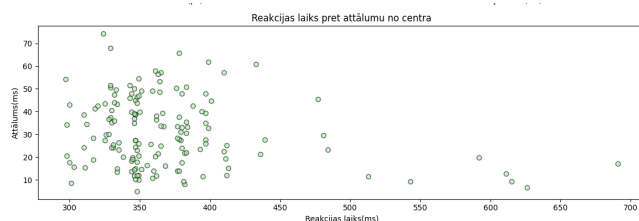
Att. 2. Attālums no centra

Figure 2. Uzrāda vairāk klišķu bija starp 20 un 30px attāluma no centra. ņemot vērā, ka $1px \cong 0.026458cm$, atbild aptuveni $0.53-0.79cm$. Novērojums, ka palielinot ātrumu “shrinking” vai mērķa grūtības(mazāks rādiuss), spēlētājs mēdz trāpīt tuvāk centrā vai tieši centrā, rezultējot precīzāka reakcija.



Att. 3. Reakcijas laiks

Figure 3. Takā koordināte nav konstante. Reakcijas laiks bieži pārsniedz vidējo $250ms$, kas tiek uzskatīts par standarta reakcijas laiku tiešsaistes testos vai ārpus tehnoloģijas. Garākie reakcijas laiki($700ms$) var būt saistīti ar to, ka rādiuss samazinās tik ātri, ka lietotājs nespēj reaģēt. Turklāt klišķināt ārpus gredzena lauka liecina par neprecīzību un vairāk “miss_count” skaits pieaug.



Att. 4. Attālums x reakcijas laiks

Figure 4, šis grafiks parāda tiešu sakarību starp reakcijas laiku un attālumu no centra. Parasti lielāka laikuma ātrāks reakcijas laiks, ja nesaktās vai ir noklišķināts tuvu apļa centram. Bet pretējā gadījumā, mazāks rādiuss izraisa mazāku precīzību. Gadījumi var būt, ka izlaiž mēģinājumu 1 no 10, lai gatavotos nākošajam, palielinot reakcijas laiku ilgāku.

IV. SECINĀJUMI

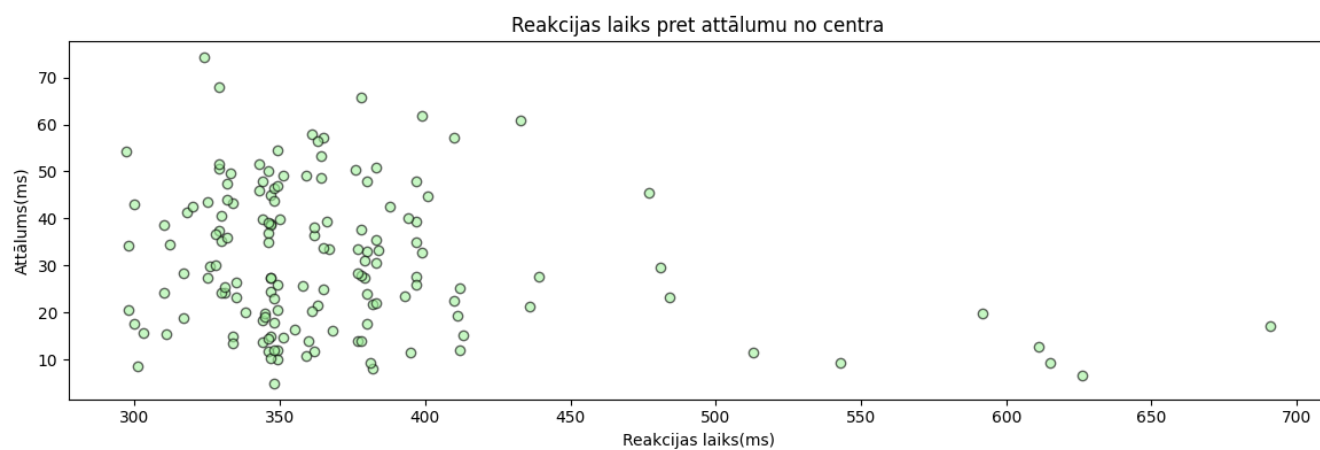
Veicot eksperimentu, tika novērtēts, ka reakcijas laiks ir atkarīgs no pieredzes(spēlētāji), kuri reglāri spēlē ritma spēles(piem., Geometry Dash,osu!, Mai Mai, u.c.), spēj reaģēt ievērojami ātrāk, dažkārt pat zem $100ms$. Turklāt reakcijas laiku ietekmē ārēji faktori, piem., datora aizture vai programmas aizkave, kas var pievienot aptuveni $30ms$ papildu kavējumu reakcijas laikam.

Lai eksperiments vai projekts būtu ticamāks, būtu nepieciešams iesaistīt vairākus lietotājus ar dažādu pieredzi, vecumu un dzimumu. Tas ļautu salīdzināt reakcijas laiku starp dažādiem lietotājiem(gan spēlētāji, gan ne spēlētāji) un samazināt individuālo faktoru ietekmi uz rezultātiem. Iegūt datus un saglabāt, piem., .xlsx vai JSON failos, lai tos varētu atkārtoti analizēt un salīdzināt. Papildus tam būtu iespējams divus atšķirīgus eksperimentus:

1. Reālas dzīves eksperiments, kur reakcijas laiks tiek mērīts ar kustīgu fizisku mērķi, ņemot vērā blakus faktorus kā gaismas pretestība, berze, gravitācija un citu vides apstākļi.
2. Digitālu eksperimentu(piem., šo pašu reakcijas laika spēli), izmantojot tiešsaistes vai video spēles reakcijas laika testus ar kustīgiem objektiem, kuros šie blakus faktori netiek tieši ietekmēti.

Papildus informāciju par līdzīgu eksperimentu[5], kur autori pēta acs, peles un ritināšanas parametru tīmekļa lapu pārlūkošanā. Mērķis bija saprast, kā vizuāli vadītas darbības un fiksācijas mijiedarbojas, analizējot kustību un fiksāciju parametrus. Lai iegūtu gala rezultātus un kopīgu parādību.

Fiksācija - mirklis, kad acs ir pieturējusies uz konkrētu punktu + informāciju/datu iegūšanu



-
- [1] pubnub, [How fast is human reaction time? human perception tech.](#)
 - [2] Pygame, [Pygame documentation.](#)
 - [3] C. Code, [Master python by making 5 games \[the new ultimate introduction to pygame\].](#)
 - [4] H. Benchmark, [Reaction time test.](#)
 - [5] frontiersin, [Similarities and differences between eye and mouse dynamics during web pages exploration.](#)

APPENDIX A

Listing 1. Circle.py “shrinking“

```

def update(self, dt):
    self.radius -= self.shrink_speed * dt
    if self.radius < 0:
        self.radius = 0

```

Listing 2. main.py Pygame tutorial

```

import pygame

pygame.init()
WINDOW_WIDTH, WINDOW_HEIGHT = 720, 720
display_surface = pygame.display.set_mode((WINDOW_WIDTH, WINDOW_HEIGHT))
running = True

while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    display_surface.fill("white")
    pygame.display.update()

pygame.quit()

```